

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	P. Harsha vardhan	Reg. No.:	19241179
Section / Batch:		Date:	17/07/2026

Code of Conduct

I P. Harsha vardhan 19241179 (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Course Coordinator

Program Director

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

identify whether the given expressions are balanced or not.

A bracket expression is balanced if every opening bracket $(, [, \{$ has a corresponding closing bracket $),], \}$ in the correct order.

i) Expression

$$[9 * (k + 8 - \{6 + 2\} + 77) / 100$$

using a stack.

Symbol	Stack
[	[
(	[(
{	[({
}	[(
)	[



At the end $[$ is still present without a matching $]$.

Therefore: Not Balanced.

Reason: opening square bracket $[$ does not have a closing $]$

i) Expression

$((a*b) + [4/2] - d)$

using a stack

Symbol	Stack
(	(
(	((
(	(((
)	((
[	(([
]	((
)	(

At the end, one opening (remains without a matching).

Therefore: Not Balanced

Reason: one opening Parenthesis (is missing its closing).

Final Answer.

1. i. Not balanced x

2. ii. Not balanced x

Compressing a stack with Repeated Elements
idea

instead of storing repeated values separately store.

(value, frequency)

Example:

Normal stack:

A A A B B C

Compressed stack

(A, 3) $\rightarrow$ (B, 2) $\rightarrow$ (C, 1)

Modified stack ADT

Push (x):

- If top value $= x$, increase its frequency by 1.

- otherwise push (x, 1).

Pop ():

- If frequency > 1 , decrease frequency by 1.

- If frequency $= 1$, remove the pair.

Top ():

- Return the value at the top

Example

Push: A, A, A, B, B

stack becomes:

TOP $\rightarrow$ (B, 2)

(A, 3)

POP once.

TOP $\rightarrow$ (B, 1)

(A, 3)

POP again:

TOP (A, 3)

Time and Space Trade-off

operation

Push

POP

TOP

Time

$O(1)$

$O(1)$

$O(1)$

Space: $O(k)$ where k is the no. of groups of different values, instead of $O(n)$ for all individual elements.

Conclusion

stack compression stores repeated elements as value + frequency. It saves memory while keeping Push, Pop, and TOP operations efficient $O(1)$.